

Statistics 6910: Relational Databases and SQL

A *database* is a collection of data with information about how the data are organized (meta-data). A *database server* is like a web server, but responds to requests for data rather than web pages.

We'll talk about *relational database management systems* (RDBMS) and how to communicate with them using the *structured query language* (SQL).

Why use a database?

- Coordinate synchronized access to data
- Change continually; give immediate access to live data
- Centralize data for backups
- Control access to the data

A RDBMS had three main parts

- Data definition
- Data access
- Privilege management

We'll concentrate on data access, assuming the database is already available and we have the needed privileges.

Topics:

- using SQL to extract info from RDBMSs
- relating these back to similar tasks in R
- using SQL from within R

There are tradeoffs in terms of what we choose to do using SQL and what we do in R.

- A database is made up of one or more two dimensional *tables*, usually stored as files on the server.
- An important concept in the design of a database is *normalization*. The idea is to remove as much redundancy as possible when creating the tables. This is done by breaking the full dataset into separate tables.
- The "relational" in RDBMS comes from the fact that we then need to link the tables together.
- For now let's talk about a single table....

- A *table* is a rectangular arrangement of values, where a row represents a case, and a column represents a variable (just like a data frame in R).
- Another term for a table is a *relation*. The rows are referred to as *tuples* and the columns as *attributes*.

ID	Test Date	Lab Results
101	2000-01-20	3.7
101	2000-03-15	NULL
101	2000-09-21	10.1
101	2001-09-01	12.9
102	2000-10-20	6.5
102	2000-12-07	7.3
102	2001-03-13	12.2
103	2000-02-16	10.1

Missing value

- An *entity* is the general object of interest. For example, a lab test. Each row (tuple) is a particular occurrence of the entity. This means that rows in the table are unique.
- To identify each row, we use a *key*. A key is just an attribute or a combination of attributes that uniquely identifies the cases.
- In the lab test example, we need a *composite key* of both patient ID and date, since neither is necessarily unique.
- In R, the row names of a data frame play a similar role.

- SQL allows us to interactively *query* the database to reduce the data by subsetting, grouping, or aggregation.
- Each database program tends to have its own version of SQL, but they all support the same basic SQL statements. (We say statements rather than commands because SQL is referred to as a declarative rather than a programming language.)
- The SQL statement for retrieving data is the *SELECT* statement. This operates on one or more tables. The result will always be another table.

We have a table called *chips*, with data about the CPU development of PCs over time

The simplest possible query gives back everything:

SELECT * FROM chips;

```

processor | date | transistors | microns | clockspeed | width | mips
-----|-----|-----|-----|-----|-----|-----
8080      | 1974 | 60000       | 10      | 2          | 8      | 0.64
8088      | 1979 | 29000       | 3        | 5          | 16     | 0.33
80286     | 1982 | 134000      | 1.5      | 6          | 16     | 1
80386     | 1985 | 275000      | 1.5      | 16         | 32     | 5
80486     | 1989 | 1200000     | 1        | 25         | 32     | 20
Pentium   | 1993 | 3100000     | 0.8      | 60         | 32     | 100
PentiumII | 1997 | 7500000     | 0.35     | 233        | 32     | 300
PentiumIII| 1999 | 9500000     | 0.25     | 450        | 32     | 510
Pentium4  | 2000 | 42000000    | 0.18     | 1500       | 32     | 1700

```

- By convention, we display SQL statements in upper case. Statements are ended by a semicolon.

Attributes / Variables

- Recall that in R, we can select particular variables (columns) by name.
`chips[, c('mips', 'microns')]` # a data frame
- The order of the variable names determines the order in which they'll be returned in the resulting data frame.
- The corresponding SQL query is

```
SELECT mips, microns FROM chips; -- a table
```

comment in SQL

Tuple/Row

In R we can select rows that match a condition:

```
chips[chips$processor == 'Pentium' | chips$processor == 'PentiumII', ]
```

The corresponding SQL statement is

```
SELECT * FROM chips
WHERE processor = 'Pentium' OR processor = 'PentiumII';
```

Note: Whitespace can be used freely in SQL statements. we will often separate lines for clarity. The statement isn't evaluated until the semicolon is entered.

- Aside: In R, it usually doesn't matter whether you use single or double quotes to surround character strings. In SQL, the standard is to use single quotes, so I will do this throughout the notes for both R and SQL.
- The WHERE clause can also be used with other Boolean operators. The keyword NOT negates a condition, and parentheses can be used to clarify order of evaluation.

```
SELECT * FROM chips WHERE date > 1990;
```

```
SELECT * FROM chips WHERE NOT width = 8;
```

```
SELECT * FROM chips WHERE NOT (width = 8 OR width = 16);
```

- In both R and SQL, we can do both types of subsetting at once.
- R:

```
chips[chips$processor == 'Pentium' |
      chips$processor == 'PentiumII',
      c('mips', 'microns')]
```

- SQL:
- ```
SELECT mips, microns FROM chips
WHERE processor = 'Pentium' OR
 processor = 'PentiumII';
```

## General Syntax

SELECT attribute(s) FROM table(s)  
[WHERE constraints];  [optional]

How would we pull the years of all 32-bit processors that execute fewer than 250 million instructions per second (mips),

1) in R,  
chips[ chips\$mips < 250 & chips\$width == 32, "date"]

2) in SQL?  
SELECT date FROM chips WHERE mips < 250 AND width = 32;

SQL offers limited features for summarizing data -- some aggregate functions that operate over the rows of a table, and some mathematical functions that operate on individual values in a row.

The aggregate functions are

- COUNT - number of rows
- SUM - total of all values for an attribute
- AVG - average value for an attribute
- MIN - minimum value for an attribute
- MAX - maximum value for an attribute

SELECT attribute(s) FROM table(s) [WHERE constraints];

 can also be *functions of attributes*

## Exercise:

1) How many rows are in the chips table?  
SELECT COUNT(date) FROM chips;

2) How many chips have attribute width equal to 32?

SELECT COUNT(date) FROM chips WHERE width = 32;

3) What is the average clock speed for the chips in question 2?

SELECT AVG(clockspeed) FROM chips WHERE width = 32;

Now answer the same questions assuming chips is a data frame in R.  
nrow(chips); sum(chips\$width == 32);  
mean(chips\$clockspeed[chips\$width == 32])

## Additional clauses: GROUP BY

- The GROUP BY clause makes the aggregate functions in SQL more useful. It enables the aggregates to be applied to *subsets* of the rows in a table.

SELECT width, MAX(mips) FROM chips  
GROUP BY width;

width | max  
-----  
8 | 0.64  
16 | 1  
32 | 1700  
(3 rows)

- More than one attribute can be included in the GROUP BY clause.

## Additional clauses: HAVING

- The WHERE clause can't contain an aggregate function, but the HAVING clause can be used to refer to the groups to be selected.

```
SELECT width, MAX(mips) FROM chips
GROUP BY width HAVING MAX(mips) >= 1;
```

```
width | max
-----+-----
16 | 1
32 | 1700
(2 rows)
```

- First the chips table is separated into sets of rows by width. For each set, MAX(mips) is calculated, and the set is discarded if MAX(mips) < 1. Finally, width and MAX(mips) are returned for each set.

## A few other predicates and clauses

- DISTINCT - forces values of an attribute in the results table to have unique values
- NOT - negates conditions in WHERE or HAVING clause
- LIMIT - limits the number of rows returned

```
SELECT * FROM chips
LIMIT 3;
```

```
SELECT DISTINCT width FROM chips;
```

## Order of Execution

The order of execution of the clauses in a SELECT statement is as follows:

1. FROM: The working table is constructed.
2. WHERE: The WHERE clause is applied to each row of the table, and only the rows that test TRUE are retained.
3. GROUP BY: The results are broken into groups of rows all with the same value of the GROUP BY clause.
4. HAVING: The HAVING clause is applied to each group and only those that test TRUE are retained.
5. SELECT: The attributes not in the list are dropped, aggregates are calculated, and options DISTINCT, ORDER BY, and LIMIT are applied.

## Using SQL with R

The DBI and RSQLite libraries allow you to connect to an SQL database, submit a query, and receive the results as a data frame.

```
library(RSQLite)
```

```
set up an interface to SQLite
m = dbDriver("SQLite")
```

```
connect to particular DBMS (no security in SQLite)
con = dbConnect(m, dbname="tfile")
```

```
submit an SQL statement; returns a data frame
dbGetQuery(con, "SELECT * FROM chipsSQLite LIMIT 5;")
```

If the results are large, we don't have to pull them all over into R at once.

```
submit an SQL statement; but keep results in SQL table
rs = dbSendQuery(con, "SELECT * FROM chipsSQLite;")
```

```
Retrieve the first 5 rows in the results
fetch(rs, n = 5)
```

```
Retrieve the next 3 rows
fetch(rs, n = 3)
```

```
Retrieve the remaining rows
fetch(rs, n = -1)
```

```
Close the query
dbClearResult(rs)
```

We can of course import the whole table, then extracting what we want using R commands.

But, it may be more efficient to use SELECT to extract and import only the results we want to work with

## Multiple Tables

- Now back to multiple tables. In this example, customers and their bank accounts are linked via the registration table:

Customers Table

| CustNo | Name     | Address   |
|--------|----------|-----------|
| 1      | Smith, J | 101 Elm   |
| 2      | Smith, D | 101 Elm   |
| 3      | Brown, D | 17 Spruce |

Accounts Table

| AcctNo | Balance | Branch |
|--------|---------|--------|
| 201    | \$12    | City   |
| 202    | \$1000  | City   |
| 203    | \$117   | City   |
| 301    | \$10    | Suburb |
| 302    | \$170   | Suburb |

Registration Table

| CID | AcctNo |
|-----|--------|
| 1   | 201    |
| 2   | 201    |
| 2   | 202    |
| 3   | 203    |
| 3   | 301    |
| 3   | 302    |

Branches Table

| Branch | Address       | Manager |
|--------|---------------|---------|
| City   | 101 Main St   | Reed    |
| Suburb | 1800 Long Ave | Green   |

- We could put this information in one big table, but this would introduce redundancy.

| CID | Name     | Address   | AcctNo | Balance | Branch | BAddr     | Manager |
|-----|----------|-----------|--------|---------|--------|-----------|---------|
| 1   | Smith, J | 101 Elm   | 201    | \$12    | City   | 101 Main  | Reed    |
| 2   | Smith, D | 101 Elm   | 201    | \$12    | City   | 101 Main  | Reed    |
| 2   | Smith, D | 101 Elm   | 202    | \$1000  | City   | 101 Main  | Reed    |
| 3   | Brown, D | 17 Spruce | 203    | \$117   | City   | 101 Main  | Reed    |
| 3   | Brown, D | 17 Spruce | 301    | \$10    | Suburb | 1800 Long | Green   |
| 3   | Brown, D | 17 Spruce | 302    | \$170   | Suburb | 1800 Long | Green   |

- There can be speed and memory advantages to working with the smaller tables, extracting just the information we need.

## Warmup

- The names and addresses of all customers.

```
SELECT Name, Addr FROM Customers;
```

|   | Name     | Address   |
|---|----------|-----------|
| 1 | Smith, J | 101 Elm   |
| 2 | Smith, D | 101 Elm   |
| 3 | Brown, D | 17 Spruce |

## Warmup

- All of the attributes in the Registration table for accounts belonging to Customer 3.

```
SELECT * FROM Registration WHERE CustNo = 3;
```

| row_names | CID | AcctNo |
|-----------|-----|--------|
| 1         | 4   | 3 203  |
| 2         | 5   | 3 301  |
| 3         | 6   | 3 302  |

## Warmup

- The account numbers for those accounts that are overdrawn in the Downtown branch.

```
SELECT AcctNo FROM Accounts WHERE
 Branch = 'City' AND Balance < 100;
```

|   | AcctNo |
|---|--------|
| 1 | 201    |

- The total balance for all accounts belonging to a branch.

```
SELECT Branch, SUM(Balance)
 FROM Accounts
 GROUP BY Branch;
```

|   | Branch | SUM(Balance) |
|---|--------|--------------|
| 1 | City   | 1129         |
| 2 | Suburb | 180          |

- The total overdrawn amount at a branch for all overdrawn accounts, for those branches which have an account with at least \$100 overdrawn.

```
SELECT Branch, SUM(Balance) FROM Accounts
 WHERE Balance < 0
 GROUP BY Branch
 HAVING Min(Balance) < -100;
[1] Branch SUM(Balance)
<0 rows> (or 0-length row.names)
```

- Suppose we want the name(s) of the owner(s) for every account
- The customer name is in the Customers
- The account number is in Registration
- For each account in Registration, we want the associated customer names.
- We need to joint two tables to get this information

## Joining Tables

```
SELECT Name, AcctNo
 FROM Customers, Registration
 WHERE Customers.CustNo =Registration.CID;
```

|   | Name     | AcctNo |
|---|----------|--------|
| 1 | Smith, J | 201    |
| 2 | Smith, D | 201    |
| 3 | Smith, D | 202    |
| 4 | Brown, D | 203    |
| 5 | Brown, D | 301    |

- Combining two tables in this way is called an *inner join*.
- Note how we refer to columns within a given table using the form `tableName.column` ("dot notation"). This isn't strictly necessary when there's only one possible match, but it improves clarity.



- We can use AS to rename tables or columns. This can save a lot of typing.

```
SELECT Name, AcctNo AS Account
FROM Customers AS C, Registration AS R
WHERE C.CustNo = R.CID;
```

|   | Name     | Account |
|---|----------|---------|
| 1 | Smith, J | 201     |
| 2 | Smith, D | 201     |
| 3 | Smith, D | 202     |
| 4 | Brown, D | 203     |
| 5 | Brown, D | 301     |
| 6 | Brown, D | 302     |

- The customer name, account number, and balance of the account for all accounts.

```
SELECT C.Name, A.AcctNo, A.Balance
FROM Customers AS C, Registration AS R,
Accounts AS A
WHERE C.CustNo = R.CID AND
 A.AcctNo = R.AcctNo;
```

|   | Name     | AcctNo | Balance |
|---|----------|--------|---------|
| 1 | Smith, J | 201    | 12      |
| 2 | Smith, D | 201    | 12      |
| 3 | Smith, D | 202    | 1000    |
| 4 | Brown, D | 203    | 117     |
| 5 | Brown, D | 301    | 10      |
| 6 | Brown, D | 302    | 170     |

- Give the total balance for each customer with multiple accounts.

```
SELECT CID, SUM(Balance)
FROM Registration AS R, Accounts
AS A
WHERE A.AcctNo = R.AcctNo
GROUP BY R.CID HAVING COUNT(*) >
1;
```

|   | CID | SUM(Balance) |
|---|-----|--------------|
| 1 | 2   | 1012         |
| 2 | 3   | 297          |

## Other types of Databases

- NoSQL
  - data bases that don't use SQL
  - Not relational
  - JSON – style data structures
- NoSQL RDBMS
  - Data base that doesn't use SQL, but is relational
  - Shell based
- Example: MongoDB (from "humongous") is a scalable, high-performance data base